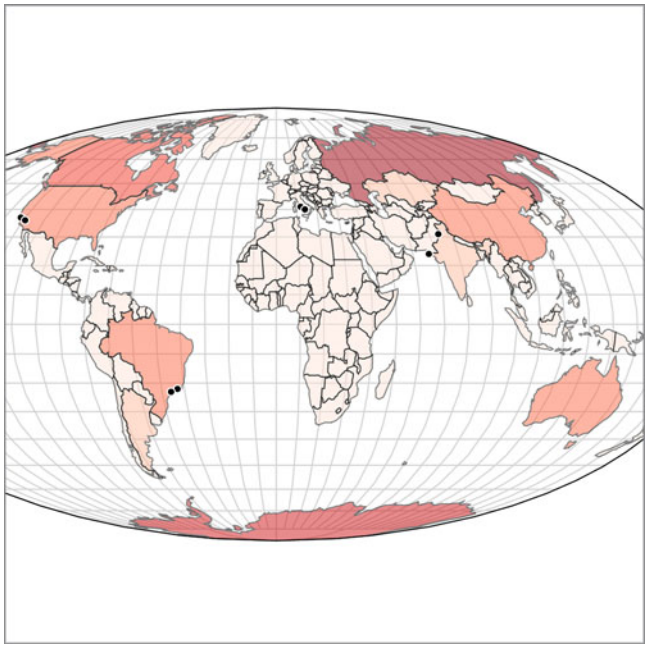


Figure 7.8 Our map with a graticule (in light gray) and a graticule outline (the black border around the edge of the map)

attribute of a `<g>` element that held our chart. This time, we'll use the scale and translate values of the zoom behavior to update the settings of our projection, which will give us the ability to zoom and pan our map.

Create a zoom behavior and call it from the `<svg>` element. Whenever you have a drag event on anything in the `<svg>`, a mousewheel event, or a double-click, then it triggers zoom. When we worked with zoom before, we only dealt with the dragging, which updates the `zoom.translate()` value and which you can use to update the translate value of whatever element you want to update. This time, we'll also use the `zoom.scale()` value, which gives us an increasing (when you double-click or roll your mousewheel forward) or decreasing (when you roll your mousewheel backward) value. To use zoom with a projection, we'll want to overwrite the initial `zoom.scale()` value with the scale value of the projection, and do the same with the zoom translate value. After that, any time we have an event that triggers zoom, we'll use the new values to update our projection, as shown in the following listing and in figure 7.9.

Listing 7.10 Zoom and pan with maps

```
var mapZoom = d3.behavior.zoom()
    .translate(projection.translate())
    .scale(projection.scale())
    .on("zoom", zoomed);

d3.select("svg").call(mapZoom);

function zoomed() {
    projection.translate(mapZoom.translate()).scale(mapZoom.scale());
```

Overwrites the translate and scale of the zoom to match the projection

Whenever the zoom behavior is called, overwrites the projection to match the updated zoom values